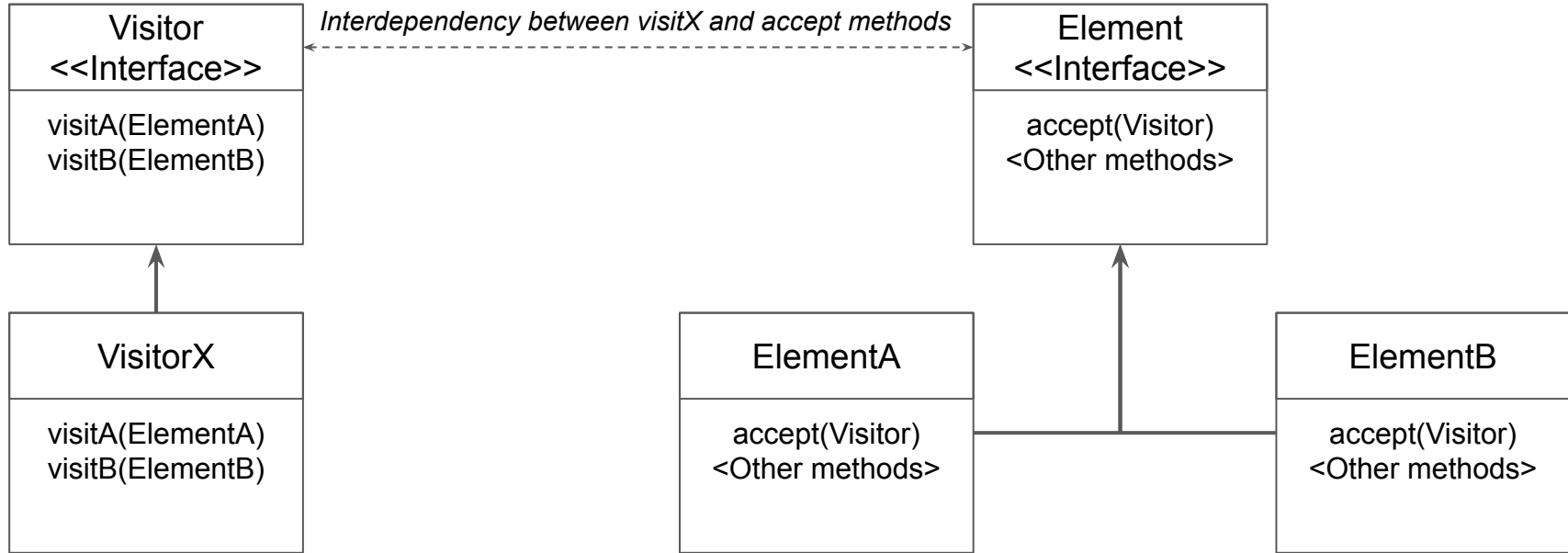


Reparación Automática de Software

Mutant Subsumption - Minimum Test Set

Pero primero revisemos el patrón visitor



Pero primero revisemos el patrón visitor

Dado cualquier objeto que herede de *Element* **e**, y un *Visitor* **v**, ejecutar el visitor sobre el elemento se reduce a la llamada **e.accept(v)**.

La ventaja es dejar la determinación de cuál método visitX llamar al tipo de **e**.

Las desventajas son:

- Alto acoplamiento, métodos asociados al patrón visitor se implementan (en parte) en los elementos a visitar.
- La cohesión se puede ver afectada, los elementos suman la responsabilidad de visita.
- Interfaces nuevas Visitor requieren modificar todos los elementos (interface y clases concretas).

Mutant Subsumption - Continuación

Para un programa original P

Dados dos mutantes m_1 y m_2 , m_1 subsume a m_2 sii para cada test t que detecta a m_1 , t también detecta a m_2 .

Es posible generalizar la definición a m_1 subsume a m_2 si para cada input/escenario x , $P(x) \neq m_1(x) \rightarrow P(x) \neq m_2(x)$

- Qué ventajas tiene la generalización?
- Qué desventajas tiene la generalización?

Mutant Subsumption - Continuación

Para un programa original P

Dados dos mutantes m_1 y m_2 , m_1 subsume a m_2 sii para cada test t que detecta a m_1 , t también detecta a m_2 .

Es posible generalizar la definición a m_1 subsume a m_2 si para cada input/escenario x , $P(x) \neq m_1(x) \rightarrow P(x) \neq m_2(x)$

Un mutante que es subsumido se dice redundante, es decir que se puede eliminar y no va a afectar a la evaluación de la test suite, o de todas las test suites en la generalización.

Mutant Subsumption - Minimum Test Set

Para un programa original **P**

Dado un conjunto de tests **TS** y un conjunto de mutantes **M** tal que

$$\forall m \in M : \exists t \in TS : \text{kills}(t, m)$$

Un conjunto de tests mínimo **MTS** $\subseteq$ **TS** cumple con la propiedad anterior (para **MTS**) y no existe **MTS'** $\subset$ **MTS** tal que la propiedad anterior se siga cumpliendo.

Dado un conjunto de mutantes **M**, y un subconjunto **M'** $\subseteq$ **M** tal que **M'** solo contiene mutantes no redundantes; si **MTS** es un conjunto mínimo de tests para **M'** entonces también lo es para **M**.

Requirement Subsumption

Dado un conjunto de tests **TS** y un conjunto de requisitos **R** tal que

$$\forall_r \in R : \exists_t \in TS : \text{covers}(t, r)$$

Un conjunto de tests mínimo **MTS** $\subseteq$ **TS** cumple con la propiedad anterior (para **MTS**) y no existe **MTS'** $\subset$ **MTS** tal que la propiedad anterior se siga cumpliendo.

Dado un conjunto de requisitos **R**, y un subconjunto **R'** $\subseteq$ **R** tal que **R'** solo contiene requisitos no redundantes; sí **MTS** es un conjunto mínimo de tests para **R'** entonces también lo es para **R**.

Dados dos requisitos r_1 y r_2 , r_1 subsume a r_2 sii para cada test t que satisface a r_1 , t también satisface a r_2 .

Preguntas

- Por qué un mutante/requisito subsumido es redundante?
- Por qué los tests que detectan todos los mutantes/requisitos no subsumidos forman un conjunto mínimo de tests?
- Por qué la propiedad general de subsunción es más fuerte pero no es utilizada?
- Si propusiera un nuevo operador de mutación para agregar a un conjunto de Mutadores Suficientes MS, cómo podría evaluar si el operador propuesto es una buena adición al conjunto MS?

Práctica

Mutation Testing framework utilizando Astroid + ast

Implemente un framework inicial para mutation testing en Python que cumpla con los siguientes requisitos (por ahora):

- Generar mutantes para al menos dos operadores de la lista propuesta en la guía práctica 3.
- Para cada mutación obtenida, generar una versión del programa original con dicha mutación aplicada.

Nota: utilice algún tipo de asistente IA para el desarrollo de esta versión original



Práctica

Spectrum Based Fault Localization (Java/Python)

Usando el proyecto en

<https://github.com/saiema/FaultLocalization/tree/introclass> complete las siguientes tareas:

- Haga correr la herramienta para alguno de los ejemplos vistos en clase.
- Implemente una herramienta similar para Python, por ahora que solo genere una versión instrumentalizada del programa original.
- Por qué la herramienta para Java provee dos métodos para contar cuando una sentencia es ejecutada: **mark** que retorna void, y **mark** que retorna un valor?